

```

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    console.log(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}

```

```

<ErrorBoundary>
  <A />
  <B />
  <C />
</ErrorBoundary>

```

Exception

```

<ErrorBoundary>
  <D />
  <E />
</ErrorBoundary>

```

```

export default function countReducer(state, action) {
  switch (action.type) {
    case "INCREMENT":
      return {
        count: state.count + action.payload
      };
    case "DECREMENT":
      return {
        count: state.count - 1
      };
    case "RESET":
      return {
        count: 0
      };
    default:
      return state;
  }
}

```

```

let initialState = {
  count: 0
};

export default function Counter() {
  let [state, dispatch] = React.useReducer(countReducer, initialState);

  return (
    <div>
      Counter
      <h1>{state.count}</h1>
      <button onClick={() => dispatch({type: "INCREMENT", payload: 5})}>Increment by 5</button>
      <button onClick={() => dispatch({type: "DECREMENT"})}>Decrement by 1</button>
      <button onClick={() => dispatch({type: "RESET"})}>Reset</button>
    </div>
  );
}

```

HEAP

```

{
  count: 0
}

```

